

온체인 상태와 내부 원장의 정합성 유지 · ReconcileService

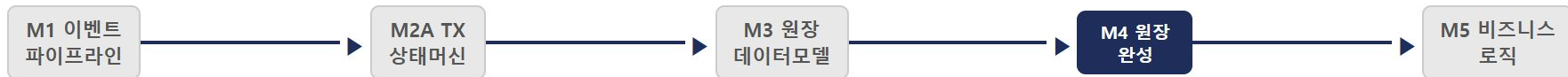
온체인 `balanceOf()`와 원장 `user_nft_holdings`가 어긋나는 이유
그리고 불일치를 탐지하고 알리는 방법

ReconcileService

원장 정합성

M4 흐름에서 S25의 위치

DAY 07 — RECONCILE + AUDIT LOG + WALLET PROVISIONING



M4 세션 흐름

- 1 S23 LedgerService — 원장 기록
- 2 S24 상태 전이 가드 — 먹등 처리
- 3 S25 ReconcileService ← 지금 — 온체인 ↔ 은행 잔액 대조
- 4 S26 AuditLogService — SHA-256 감사 체인

S25가 다루는 것

- totalSupply ↔ 수탁 계좌 1:1 불변식
- 불일치 탐지 + 알림 설계
- onMint / onBurn 이벤트 핸들러

S25가 다루지 않는 것

- 온체인 자동 수정 (절대 금지)
- 원장 재발행 (운영팀 판단 사항)
- 원화 출금 처리 (→ M6)

핵심 전제 — 온체인 balanceOf()가 진실. 원장은 파생 캐시. ReconcileService는 두 상태의 불일치를 탐지하고 알리는 감시자.

원장과 온체인이 어긋나는 상황 — 3가지 시나리오

S24 맥등 처리 이후에도 해결 안 된 문제

시나리오 1 — Consumer 장애

- NFTIssued 이벤트 수신
- DB 처리 중 서버 순간 장애
- Worker crash → PEL 잔류
- 재시작 후 DLQ 처리 지연

온체인: balanceOf = 1
원장: holdings 없음

시나리오 2 — 블록 Reorg

- 블록 #1000에서 NFTIssued 처리 완료
- 체인 Reorg → 블록 #1000 폐기
- TX 소실 — 원장에만 기록 남음

온체인: NFT 없음
원장: NFT 보유 기록 있음

시나리오 3 — 운영자 실수

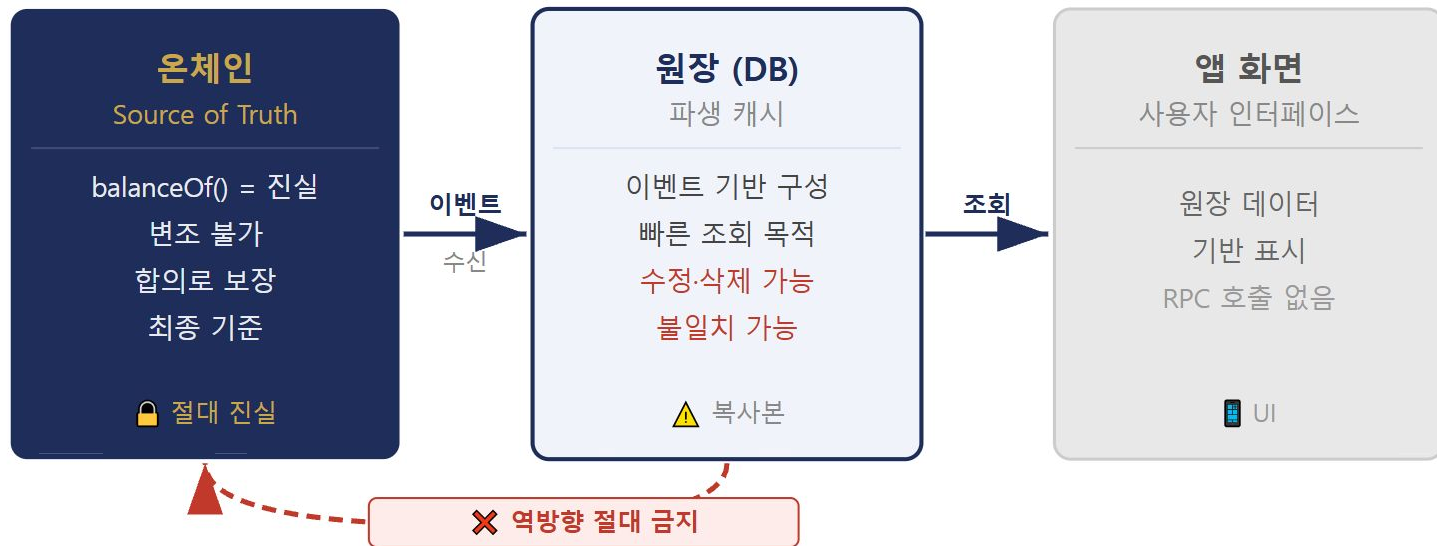
- DB 직접 접속
- user_nft_holdings 레코드 잘못 삭제

온체인: balanceOf = 1
원장: NFT 없음

공통 결과 — 불일치가 쌓이면 원장을 신뢰할 수 없게 된다. 사용자가 앱을 열면 "내 NFT가 없다"는 화면이 뜬다. 온체인에는 분명히 있는데. **이 문제를 탐지하는 것이 ReconcileService의 존재 이유다.**

단일 진실 원칙 — 온체인이 항상 맞다

Source of Truth vs 파생 캐시 — 역방향 금지의 근거



핵심 결론 — 불일치 발생 시 원장을 온체인 기준으로 보정한다. 반대 방향(원장 → 온체인 수정)은 절대 금지. 이 방향이 열리면 **DB 권한 = 온체인 자산 발행 권한**이 되어 버린다.

역방향 금지 — 절대 원칙

RECONCILE SERVICE가 할 수 있는 것 vs 없는 것

✅ ReconcileService가 하는 것

- balanceOf(addr, tokenId) 조회
ERC-1155 온체인 상태만 읽음 — 쓰기 없음
- user_nft_holdings SELECT
DB 읽기 전용 — INSERT/UPDATE/DELETE 없음
- LEDGER_ONLY / ONCHAIN_ONLY 분류
원인별 대응이 다르므로 구분 필요
- isHealthy 판정
discrepancies.length === 0 이면 true
- alerter.fire() 호출
알림 발송 후 역할 끝 — 보정은 담당자 몫

❌ ReconcileService가 절대 안 하는 것

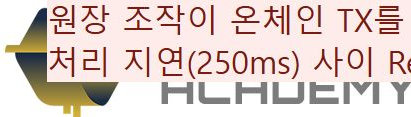
- 원장 자동 보정 (강제 INSERT)
처리 지연인지 진짜 불일치인지 코드가 판단 불가
- 온체인 mint / burn TX 발행
원장 → 온체인 방향 = 보안 공격 경로
- 불일치 자동 해소
Reorg 구간에서 자동 보정 시 결과 예측 불가
- 원인 판정 없이 DB 직접 수정
원인 분석은 반드시 사람이 — 코드가 하면 안 됨

왜 자동 보정을 금지하는가?

Reorg 중에는 노드마다 다른 상태를 반환 → Reconcile 결과 자체가 신뢰 불가.

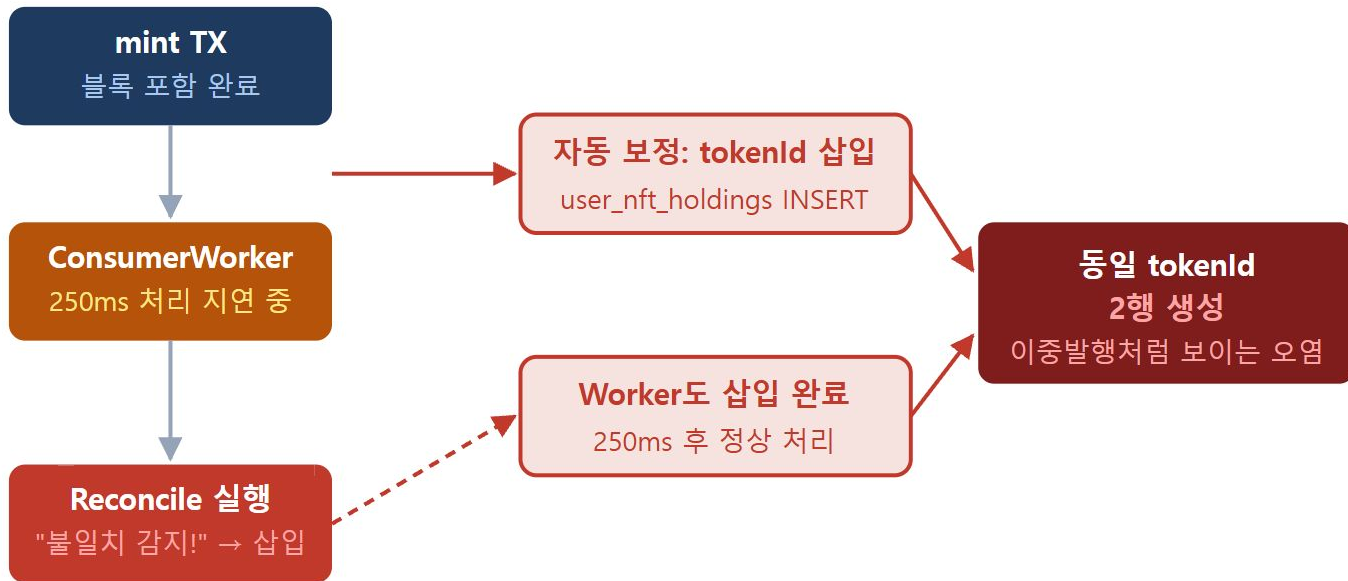
원장 조작이 온체인 TX를 유발하면 → 이것이 보안 공격 경로가 됨.

처리 지연(250ms) 사이 Reconcile 실행 → 이중 발행 데이터 오염.



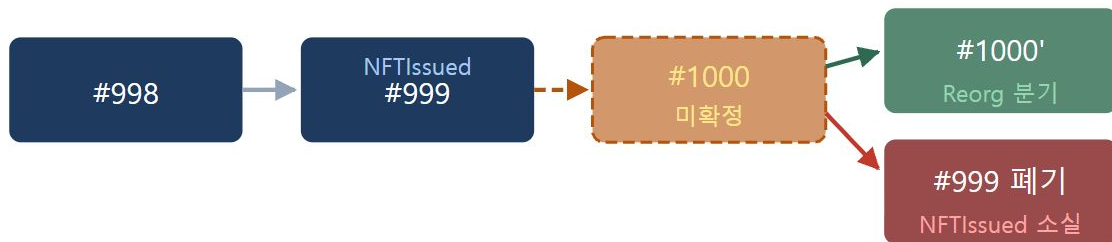
자동 보정 금지 — 케이스 A: 정상 지연 오인

코드는 "지연 중"과 "진짜 누락"을 구분할 수 없다



자동 보정 금지 — 케이스 B: Reorg 구간

온체인 상태가 확정되지 않은 순간엔 Source of Truth도 흔들린다



Reconcile 이 순간 실행

노드 A: NFT 있음 반환 / 노드 B: NFT 없음 반환 → 어느 쪽이 진실?

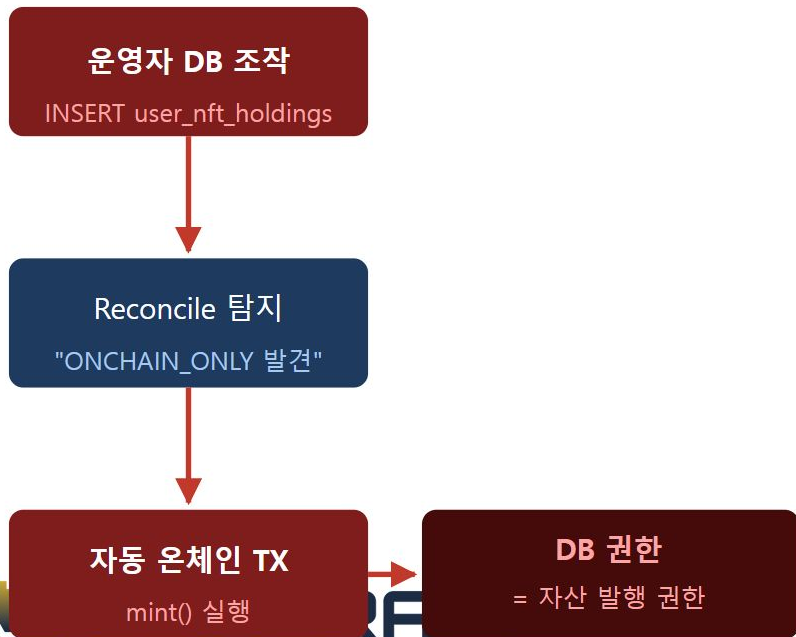
노드 A 기준 보정: 원장에 NFT 삽입
→ Reorg 확정 시 원장 오염

노드 B 기준 보정: 원장에서 NFT 삭제
→ 정상 확정 시 자산 소실

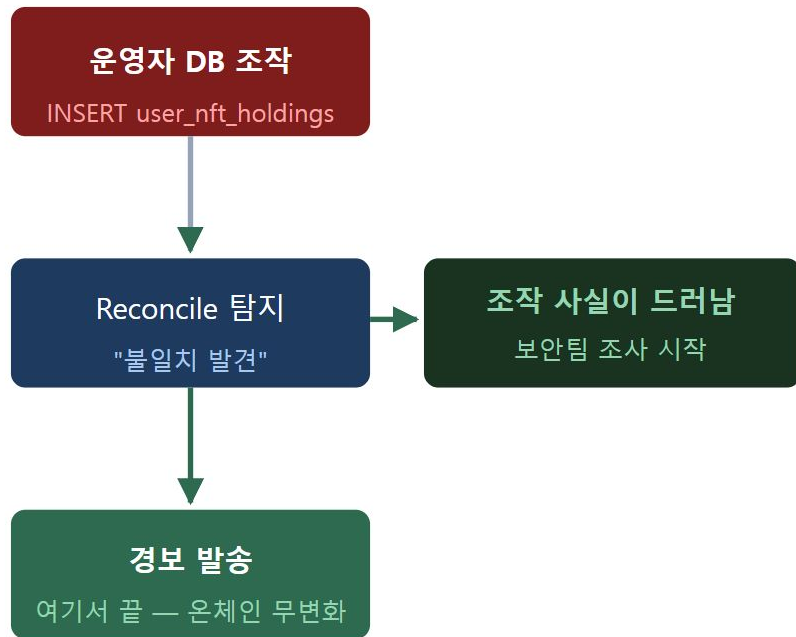
자동 보정 금지 — 케이스 C: 보안 공격 경로

자동 보정의 존재 자체가 DB 접근 권한을 자산 발행 권한으로 만든다

자동 보정 있을 때



자동 보정 없을 때



IReconcileService — 전체 인터페이스

RECONCILESERVICE.TS — 탐지 전용 설계 원칙

메서드	Phase	역할
<code>reconcile()</code>	1	KRW 총발행량 vs 수탁 잔액 대조 → warn / critical 분류
<code>getLastResult()</code>	1	최근 대조 결과 인메모리 캐싱 (재실행 없이 조회)
<code>reconcileNftHoldings()</code>	1	단일 사용자 NFT 원장 ↔ 온체인 대조
<code>reconcileAllNftHoldings()</code>	1	전 사용자 일괄 배치 대조
<code>onMintEvent()</code>	2	Mint 이벤트 수신 → 증분 대조 트리거
<code>onBurnEvent()</code>	2	Burn 이벤트 수신 → 증분 대조 트리거

설계 원칙 — `mintBatch` · `forceMint` · `sendTransaction` 없음. `ReconcileService`는 탐지만 한다. 보정은 운영자가 별도 채널(DLQ 재처리·이벤트 재발행)로 수행.

NftReconcileResult — 타입 설계

RECONCILESERVICE.TS — NFT 대조 결과 인터페이스

필드	의미	예시
isHealthy	불일치 0건이면 true	false → alerter.fire() 호출
discrepancies[]	불일치 목록 (userId + tokenId + type)	[{userId:'u-1', tokenId:42n, type:'ONCHAIN_ONLY'}]
checkedAt	대조 실행 시각	Date.now()

severity 기준 — 불일치 1~9건: **warn** / 10건 이상: **critical** . 단건도 isHealthy = false.

ReconcileService 생성자 — 의존성 주입

4가지 의존성: COREBANKING + ONCHAIN + LEDGER + ALERTER

의존성	메서드	역할
coreBanking ICoreBankingAdapter	<code>getUserAccount(userId)</code>	사용자 지갑 주소(walletAddr) 조회 — NFT 대조 시 온체인 주소 확인
	<code>recordTransaction(tx)</code>	KRW_MINT · KRW_BURN 이벤트를 감사 원장에 기록
onchain 블록체인 RPC	<code>getTotalSupply()</code>	KRW 스테이블코인 온체인 총발행량 조회
	<code>getCustodyAccountBalance()</code>	교보생명 원화 수탁 계좌 잔액 조회
	<code>balanceOf(addr, tokenId)</code> <code>getNftHoldings(addr)</code>	ERC-1155 NFT 보유 수량 및 보유 목록 조회
ledger 내부 원장 DB	<code>getHoldings(userId)</code>	원장 기준 사용자 NFT 보유 tokenId 목록 조회
	<code>getAllUserIds()</code>	전체 사용자 목록 — <code>reconcileAllNftHoldings()</code> 배치 대조용
alerter 알림 시스템	<code>fire(message, severity)</code>	불일치 감지 시 warn / critical 알림 발송 (PagerDuty · Slack 등)

reconcileNftHoldings() — 핵심 구현

원장 holdings vs 온체인 balanceOf — 사용자별 비교

- 1 ledger.getHoldings(userId) — 원장 NFT 보유 tokenId 목록 조회
- 2 coreBanking.getUserAccount(userId) — 온체인 지갑 주소(walletAddr) 조회
- 3 원장 각 tokenId → onchain.balanceOf(addr, tokenId) → 0이면 **LEDGER_ONLY**
- 4 onchain.getNftHoldings(addr) → 원장에 없는 tokenId → **ONCHAIN_ONLY**
- 5 불일치 ≥ 10 건 → **critical** / 1~9건 → **warn** / 0건 → isHealthy: true

순차 루프 이유 — balanceOf() RPC 호출마다 네트워크 왕복. Rate Limit 초과 방지를 위해 순차 처리. 실제 운영에서는 batchSize로 청크 처리.

severity 분류 — 불일치 건수 기준

단건도 isHealthy = false — 건수로 심각도를 판정한다

severity 판정 로직

- 불일치 0건 → **isHealthy: true**, 알림 없음
- 불일치 1~9건 → **warn**, Slack 알림
- 불일치 10건 이상 → **critical**, PagerDuty 즉시

10건 임계값 이유

- 1~2건: 단발 Consumer 장애, 곧 복구 가능
- 10건 이상: 시스템 수준 문제 (DLQ 쌓임 등)
- 임계값은 운영팀 결정 — 코드 상수로 분리 권장

불일치 건수	isHealthy	severity	조치
0건	true	없음	정상 운영
1~9건	false	warn	Slack 알림 → 담당자 확인
10건 이상	false	critical	PagerDuty → 런북 즉시 실행

LEDGER_ONLY vs ONCHAIN_ONLY 심각도 차이 — LEDGER_ONLY(원장 과잉)가 더 위험. 원장에 없는 자산이 기록됐다는 것은 Reorg 또는 DB 조작 의심. ONCHAIN_ONLY는 이벤트 지연으로 자연 해소될 수 있음.

실행 주기 — 실시간 + 정기 이중 검증

두 경로가 함께 작동해야 누락이 없다

실시간 경로

- 1 스마트 컨트랙트 `emit Mint(amount, txHash)`
- 2 `ChainEventListener.onLog()` 수신
- 3 `ReconcileService.onMint(amount, txHash)` — 즉시 기록

정기 경로

- 1 매일 00:00 KST cron 실행
- 2 `ReconcileService.reconcile()` — 전체 대조 → isHealthy 판정 → 불일치 시 `alerter.fire()`

NestJS @Cron 예시

```
@Cron('0 0 * * *', { timeZone: 'Asia/Seoul' })
async dailyReconcile(): Promise<void> {
  await this.reconcileService.reconcile();
}
```

node-cron 예시

```
cron.schedule('0 0 * * *', async () => {
  await reconcileService.reconcile();
}, { timezone: 'Asia/Seoul' });
```

NFT holdings Reconcile — 사용자별 온체인 비교

KRW 집계 비교와 달리 개별 사용자·토큰 단위 검증

```
// 불일치 유형 - 2가지
type DiscrepancyType =
  | 'LEDGER_ONLY' // 원장O / 온체인X → Reorg·DB 조작
  | 'ONCHAIN_ONLY'; // 온체인O / 원장X → 이벤트 미처리

async reconcileNftHoldings(userId: string) {
  // 1. 원장: user_nft_holdings WHERE user_id=$1
  // 2. 지갑 주소: user_wallets WHERE user_id=$1
  // 3. 각 tokenId 순회:
  //   ERC-1155 balanceOf(userAddress, tokenId)
  //   balance===0n → LEDGER_ONLY 추가
  // 4. 온체인에만 있는 것 → ONCHAIN_ONLY 추가
  // 5. 불일치 >= 10건 → critical, 미만 → warn
  return { isHealthy, discrepancies, checkedAt };
}
```

유형	의미	원인
LEDGER_ONLY	원장O, 온체인X	Reorg / DB 직접 조작
ONCHAIN_ONLY	온체인O, 원장X	Consumer 장애 / 이벤트 미처리

KRW reconcile vs NFT reconcile

KRW: totalSupply vs bankBalance — 집계 1회 비교

NFT: 사용자별 × 토큰별 — 개별 RPC 호출 반복

실행 주기 차이

KRW reconcile: 매일 00:00 KST (최우선)

NFT reconcile: 매시 또는 이벤트 트리거

사용자 증가 시 배치 + 페이지네이션 필요

Reconcile 스케줄링 — 두 계층 설계

전체 순회 비용 vs 실시간 감지 — Hourly + Daily 분리

계층	실행 주기	대상	비용	목적
Hourly	매시간 정각	최근 1시간 내 변경된 사용자만	낮음	이벤트 미처리 조기 감지 (Consumer 누락·지연)
Daily	새벽 3시 KST	전체 user_nft_holdings 순회	높음	누적 오차 감지 (Reorg·수동 DB 조작·코드 버그)

상황	Hourly	Daily
Consumer 누락 (이벤트 처리 지연)	1시간 내 감지	감지 (늦음)
Reorg 후 원장 미업데이트	대상 아닐 수 있음	전체 순회 감지
수동 DB 조작	최근 변경이면 감지	반드시 감지
코드 버그로 전체 오차	범위 밖	감지

사용자 10,000명 × eth_call = 10,000 RPC 호출 → Daily만으로 비용 집중. Hourly는 변경된 사용자만 → 실시간성 확보.

불일치 알림 설계 — 건수별 단계 + 원인 분류

건수에 따라 대응 채널과 긴급도가 달라진다

건수	심각도	알림 채널	의미 / 조치
1~4건	P3	담당자 Slack DM	단발성 이슈. 로그 조회 → 원인 파악 → 이벤트 재처리
5~9건	P2	팀 채널 @ 태그	반복 누락 가능성. Consumer 상태 + DLQ 조회
10건+	P1	즉시 에스컬레이션	시스템 전반 장애 가능성. 전체 파이프라인 점검 + 인시던트 생성

원인	증상	확인	조치
A. Consumer 누락	DLQ에 해당 txHash 존재	GET /admin/dlq/pending	DLQ requeue
B. Reorg 미처리	원장 있음, 온체인 없음	getReceipt(txHash) = null	원장 레코드 무효 처리
C. 수동 DB 조작	감사 로그에 직접 쿼리 이력	DB 접근 로그 + 감사 로그	보안팀 보고
D. 코드 버그	특정 이벤트 타입 반복	불일치 사용자 × 이벤트 패턴	코드 수정 + 배포

에스컬레이션 절차 — 불일치 발견 → 보정 → 재확인

보정은 온체인 기준으로, 역방향 수정 절대 금지.

- 1 불일치 감지 → reconcile_history 자동 기록 + 건수별 알림 발송
- 2 원인 분류 — DLQ 교차 검색 / getReceipt 확인 / DB 접근 로그 조회 / 이벤트 패턴 분석
- 3 보정 방향 확인 — **온체인 기준으로 원장 보정**. 원장 기준으로 온체인 TX 발행 금지 ✖
- 4 보정 실행 (2인 확인) → AuditLogService.log() 기록 필수
- 5 POST /admin/reconcile/run → mismatch: false 확인 후 종료

역방향 수정 금지 재확인 — 보정 코드에 mintBatch() · mint() · sendTransaction() 있으면 즉시 중단.

온체인 balanceOf = 0 → 원장 레코드 무효 처리 ✅ | 원장 있음 → 온체인 mint 발행 ✖

불일치 대응 런북 — Step 1~3

CRITICAL 알림 수신 시 즉시 실행

Step 1 · 알림 확인

disparity 방향 확인

- 양수 → 과잉발행
- 음수 → 과소발행

발생 시각 + txHash 기록

Step 2 · 이벤트 백로그 확인

ChainEventListener 큐 확인

- 미처리 Mint/Burn 있나?
- DLQ 메시지 있나?

처리 중이면 5분 대기 후 재대조

Step 3 · 감사 로그 추적

`AuditLogService.queryByResource()`

- KRW_MINT / KRW_BURN 기록 조회
- 누락된 txHash 찾기

onchain explorer와 교차 확인

Step 1~3는 읽기 전용 조사. DB 수정 또는 온체인 TX 발행은 Step 5 원인 판정 이후. 조사 전에 수정하면 증거가 사라진다.

불일치 대응 런북 — Step 4~6

원인 판정 → 수정 → 완료

Step 4 · 온체인 직접 조회

블록 탐색기에서 totalSupply() 직접 확인

CoreBanking API에서 잔액 직접 조회

두 값이 Reconcile 알림과 일치하는지 교차 검증

Step 5 · 원인 판정

이벤트 미처리 → Consumer 재시작

Reorg → 블록 확정 후 재대조

DB 조작 → 보안팀 에스컬레이션

API 오류 → CoreBanking 팀 연락

Step 6 · 완료 기록

AuditLogService.log()로 조치 내역 기록

Reconcile 재실행 → isHealthy: true 확인

인시던트 보고서 작성 (금융기관 의무)

S25 완료 기준 — 정상 대조 → isHealthy: true / disparity ± 2 → warn / disparity > 100만 → critical / onMint→KRW_MINT 기록. 7개 테스트 전부 PASS.

실습 1 — ReconcileAdminService 스켈레톤

src/admin/ReconcileAdminService.ts · 4개 의존성 주입 → 3개 메서드 구현

의존성	타입	역할
db	DatabaseClient	user_nft_holdings 쿼리 · reconcile_history 저장
reconcileService	ReconcileService	S25 구현체 — reconcileNftHoldings() 호출
auditLog	AuditLogService	수동 트리거·불일치 감사 로그 기록
notifier	NotifierAdapter	P1/P2/P3 알림 발송

메서드	대상	TODO 핵심
runHourlyReconcile()	최근 1시간 변경 사용자	updated_at >= NOW() - INTERVAL '1 hour' 쿼리
runDailyReconcile()	전체 사용자	SELECT DISTINCT user_id FROM user_nft_holdings
runManualReconcile(userId, operator)	단일 사용자	auditLog.log(RECONCILE_MANUAL_TRIGGER) + reconcile 실행

_runReconcileForUsers() 공통 로직 — reconcileNftHoldings() 루프 → 불일치 시 RECONCILE_MISMATCH_DETECTED 기록 → reconcile_history INSERT →
_sendMismatchAlert() 호출

실습 2 — ReconcileAdminRouter 스켈레톤

src/admin/ReconcileAdminRouter.ts · 2개 엔드포인트 구현

엔드포인트	입력	TODO
GET /admin/reconcile/history	query: limit, runType	reconcile_history 조회 → { history: [...] } 응답
POST /admin/reconcile/run	body: userId, operator, reason	유효성 검사(400) → runManualReconcile() → { mismatch, mismatchCount, durationMs }

history 응답 예시

```
{ "history": [{  
  "run_type":      "HOURLY",  
  "target_count": 42,  
  "mismatch_count": 1,  
  "mismatch_user_ids": ["user-007"],  
  "duration_ms": 12450  
}]}
```

run 응답 예시

```
{  
  "userId":      "user-007",  
  "mismatch":    false,  
  "mismatchCount": 0,  
  "durationMs": 843  
}
```


실습 3 — cron 등록 스켈레톤

src/admin/index.ts · node-cron 두 스케줄 등록

스케줄	cron 식	timezone	오류 시 알림
Hourly	'0 * * * *'	Asia/Seoul	P2
Daily	'0 3 * * *'	Asia/Seoul	P1

- 1 node-cron import → cron.schedule(식, 핸들러, { timezone })
- 2 핸들러 내부: 시작 로그 → reconcileAdmin.runXxxReconcile() → 완료 로그 출력
- 3 오류 발생 시 notifier.sendAlert() — Hourly=P2, Daily=P1
- 4 완료 로그 포함 항목: 대상 N명, 불일치 N건, 소요 Nms

완료 기준 — Hourly 168회/주 · Daily 7회/주 정상 실행 / cron 실패 시 P1-P2 알림 발송 확인 / `grep mintBatch ReconcileAdminService.ts` → 결과 없음 (역방향 수정 금지)

S25 완료 → S26 AuditLogService

다음: 금융 규제 대응 감사 로그 — SHA-256 체크섬과 불변성 보장

✅ S25에서 배운 것

KRW 스테이블코인 1:1 불변식
ReconcileResult 타입과 설계
tolerance $\pm 1n$ / severity 분류
onMint / onBurn 이벤트 핸들러
역방향 수정 절대 금지 원칙

→ S26에서 배울 것

금융 규정: 전자금융감독규정 §34, 가상자산법 §15
Append-only 원칙 (INSERT만 허용)
SHA-256 checksum 생성 원리
log() / verifyIntegrity() / queryByResource()
DB RLS (Row Level Security)

M4 세션	핵심 산출물	상태
S23 LedgerService	원장 기록 서비스	완료
S24 상태 전이 가드	역등 이벤트 처리	완료
S25 ReconcileService	NFT holdings 대조	완료
S26 AuditLogService	SHA-256 감사 체인	다음